

# FenixOS / AnonNet — Raport Bezpieczeństwa v007

Zdecentralizowana sieć P2P z łańcuchem FNX (monero-style), Fenix OS Live ISO z amnezją, własnym GUI, komunikatorem E2E i rdzeniem ukrytych kwot. Raport z pełnego audytu wykonanego na tym snapshocie kodu (nie na obietnicach).

**WERDYKT: WSZYSTKO ZIELONE — 36/36 selftesty · 21/21 kontroli statycznych**

red-team self\_attack: 27 ataków odparzonych · 5 notatek · 0 problemów · build\_iso: 57/57 · data: 2026-08-06 19:20 · commit: 2e5addd D47+D48: TX\_BAN\_EVT 0x06 (tombstone k-z-n, wyk  
środowisko audytu: python 3.13.14 · uid=1000, bez roota, bez qemu (Linux kontener)

## Co znaczy „zielone” uczciwie

Kod SAM siebie atakuje i broni: każdy moduł ma selftest, a full\_audit uruchamia wszystkie 36 jako osobne procesy + 21 kontroli statycznych (sekrety, exec-bity, .gitignore, dokumentacja, czystość drzewa git). To NIE dowód na 'brak 0day' — to dowód, że zaimplementowane mechanizmy obronne naprawdę działają (a dziury, które znaleźliśmy po drodze, mają test-strażnik). Sekcja problemów niżej mówi wprost, czego jeszcze nie ma.

# Jak cała sieć działa razem (warstwy 0-7)

## Warstwa 0 TOŻSAMOŚĆ

core/identity + core/keystore: username + passkey (Argon2id) → Ed25519+X25519 w kontenerze KS1; passkey-panika otwiera wabik (D24); adres FNXS1 = sig||x pub + checksum (jeden codec core dla portfela i messenger).

## Warstwa 1 OS (FenixOS Live ISO)

amnezja (tmpfs, live-build), spoof MAC/hostname/machine-id PRZED siecią (network-pre), nftables FAIL-CLOSED (policy drop x3, skuid 1088, fwmark 51820), panicd D27 (Del+PageUp 2 s → shred+poweroff), sealed payload (Shamir 3z5), tmpfiles /run/fenix 0770 na pliki sterujące RAM.

## Warstwa 2 TRANSPORT

net/frame AEAD v2 (chaCha20-Poly1305, nonce per SEQ, replay-okno; drut = zero jawnych markerów — test), camo A (rekordy 4096B + cover), dnscamo D (base32 qname↔TXT), Mullvad WG (kill switch PRZED tunelem; konto tylko z env).

## Warstwa 3 SIEĆ P2P

net/fenix\_node: mesh gossip tx/bloków (flood anti-echo), sync z chunkingiem, PoW CPU (argon-lite/sha256d), fork-choice najdłuższy ważny łańcuch; demon wystawia IPC /run/fenix/node.ipc (0660, NDJSON, biała lista opów, limity).

## Warstwa 4 CHAIN

chain/ledger: konta+nonce (chain+mempool), fee D15 (0.0005% burn + 0.055% → KOPACZ bloku w coinbase, D39; skarbiec tylko z usług rejestru), retarget 144 bloków, mempool cap; username on-chain (D28); v2: pool wpisów v2 bez kwot (C+blob), dedup ki wspólny v1+v2; chain.dat (D42, v005): zrzut atomowy + replay pełnym konsensusem (plik = obcy łańcuch); \*\*TX\_CONTACT\_REF (D45, v006): nick→wallet→kontakt FNXS1 (P21)\*\* + \*\*TX\_RANK\_UP (D46, v006): rangi z konsensusu — upgrade=dopłata różnicy, 6 confs, selite/fenix systemowe\*\* + \*\*TX\_BAN\_EVT 0x06 (D47, v007): tombstone k-z-n — kropka  $\geq 3/5$  Ed25519 (pub||x→wallet bind), kod ban\_codes + powód PL/EN  $\leq 280$  + hash dowodów; zbanowany: zero tx/coinbase, mempool czyszczony; wykup = 1/historia → skarbiec (D30, kwota robocza) i re-ban na zawsze (D18); replay bit-w-bit\*\*.

## Warstwa 5 PRYWATNOŚĆ

stealth (odbiorca niewidoczny), LSAG ring 5-11 (nadawca w tłumie), key-image anty-double-spend, pool z retencją wydanych jako wabiki; Pedersen+Borromean 64-bit; \*\*3b CLSAG + KONSENSUS v2 (tx\_hidden, D38, v003)\*\*: MLSAG [P, D=C-Σout-fee·H] + blob AEAD; bramka SHIELD v2 (amt publiczne jak Zcash t→z, C==commit(amt,r\_pub), suma==spalenie); RING v2 = pełna mgła w ledgerze (payload i pool = zero liczb, test!); migracja 3a żywa; P0-inflacja v1 multi-out załatana. \*\*v004 (D39/D41):\*\* UNSHIELD v2 0x12 (wyjście mgła→konto, public Y·H jak Zcash z→t; reszta wraca do mgły) zamyka pętlę wartości; pseudoOut per wejście = poprawne MLSAG wielo-wejścia (złapane testem M7d); GUI pool v2: mint/scan C/blob/send/unshield.

## Warstwa 6 APP/GUI

gui/fenix\_gui (dashboard/portfel/pool/kontakty (resolver + czat E2E M4)/sieć/ustawienia, suwak paranoi D29, badge demon), gui/backend\_ipc (most NDJSON z backoffem), gui/pool\_panel (adres FNXS1, TX\_RING z okna, reszta do siebie; v2: mgła — wtop/send/wypłata z panelu, 9/9), gui/panic\_button (hold-2 s D27 + czysty ekran Shift+Esc + vpn.env 0600 w RAM).

## Warstwa 7 KOMUNIKACJA M4

app/messenger: kontakt FNXS1, fp TOFU jest tożsamością (username = etykieta), E2E X25519→HKDF→ChaCha + podpis ed25519, seq anty-replay; TofuStore 0600+shred + \*\*walizka TF1 (D40, v004): kontakty szyfrowane hasłem właściciela (Argon2id+ChaCha20-Poly1305, AAD=b"TF1/tofu"; złe hasło → odmowa; legacy plaintext czytany)\*\*; okno czatu w GUI Kontakty (test 16: drut bez plaintextu, TOFU pin, replay drop, podszywka = OSOBNY kontakt); \*\*mesh T\_MSG (D37, v002)\*\*: ramka 0x22, gossip ttl=5, dedup blake2s(canon(env)), skrzynka RAM per fp (max 64 fp x 200), relay nic nie trzyma (test), chunking T\_SYNC\_PART, IPC msg\_sub/send/poll + MslpTransport (GUI ON-LINE=mesh / OFF-LINE=lokalny); \*\*offline-buffer (D44, v005): spool RAM+TTL 1h dla nieobecnych fp, powrót = msg\_sub opróżnia\*\*; \*\*FNX-R1 ratchet (D43, v005): koperta v2 ukrywa nadawcę, mk 1x+wymazanie, kick co kierunek, PFS testowany złodziejem\*\*; \*\*v006: czat pisze po NICKU (auto-rozpoznanie FNXS1 vs on-chain ref; profil pokazuje rangę z konsensusu + pending x/6)\*\*.

## Warstwa 8 OBRONA (M6)

ai/ai\_sentinel (D48, v007): radar fail-open jednego node'a — karty zdarzeń TYLKO meta (text/payload = odmowa D5), kapsuła RAM ring-buffer + purge (spalenie klucza sesji), progi wersjonowane z ban\_codes §4 (flood 0x11 / bad-TAG

0x12 / replay 0x15 / hello-farm 0x16), Safe Harbor (sync/ping osobno, zegar  $\pm 5$  min), wzorce-nie-zdarzenia (bramki PATRZE), decay (throttle 7 dni, wpis 30 dni, peer znika po dobie); sufit reakcji = wniosek-do-kropki-k-z-n (Al NIGDY nie banuje/destruuje, D17: tylko owner/admin); suwak telemetry off/minimal/full; TEST E2E: zalew→red→kropka 3/5→TX\_BAN\_EVT→tombstone.

## Selftesty: 36/36 zielone (czas, rc)

```
✓ [CORE      ] identity.py                                0.1s rc=0
✓ [CORE      ] keystore.py (KS1/panic/backoff)           0.3s rc=0
✓ [CORE      ] crypto/fenix_crypto.py (E2E+Shamir)       0.1s rc=0
✓ [CORE      ] crypto/fenix_wrap.py                     0.9s rc=0
✓ [CHAIN     ] pow.py (PoW argon-lite)                  2.1s rc=0
✓ [CHAIN     ] battery: stealth/ring/amount (M5b 1-3)    1.6s rc=0
✓ [CHAIN     ] ring_sig.py (nadawca)                    1.5s rc=0
✓ [CHAIN     ] amount_hide.py (kwoty)                   3.7s rc=0
✓ [CHAIN     ] miner.py                                  0.1s rc=0
✓ [CHAIN     ] ledger.py (konsensus)                    0.1s rc=0
✓ [CHAIN     ] tx_ring.py (pool prywatny E2E)           2.8s rc=0
✓ [CHAIN     ] usernames.py (D28 on-chain)              0.1s rc=0
✓ [NET       ] frame.py (AEAD v2/glina)                 0.1s rc=0
✓ [NET       ] fenix_node.py (mesh+mining+drut)         5.2s rc=0
✓ [TRANSPORT] camo.py (padding/cover)                   1.4s rc=0
✓ [TRANSPORT] dnscamo.py (D33 loopback)                 9.0s rc=0
✓ [TRANSPORT] mullvad.py (kill switch)                  0.1s rc=0
✓ [OS        ] spoof.sh --selftest                      1.1s rc=0
✓ [OS        ] fenix_panicd.py (D27)                    0.0s rc=0
✓ [OS        ] fenix_boot.py (bariera D24)              0.2s rc=0
✓ [OS        ] fenix_payload.py (sealed)                0.1s rc=0
✓ [OS        ] build_iso.sh --selftest (57)             1.7s rc=0
✓ [GUI       ] fenix_gui.py (D28/D29 + panic + vpn.env + IPC + Pool + czat E2E) 7.2s rc=0
✓ [GUI       ] panic_button.py (D27 w GUI, hold-2s)     0.1s rc=0
✓ [GUI       ] backend_ipc.py (most GUI+demon, E2E)     0.8s rc=0
✓ [GUI       ] pool_panel.py (M7c: TX_RING z GUI, E2E)  52.7s rc=0
✓ [APP       ] messenger.py (M4: E2E+TOFU+impersonacja) 0.2s rc=0
✓ [CHAIN     ] clsag.py (3b rdzeń: ukryte kwoty MLSAG)  10.2s rc=0
✓ [CHAIN     ] tx_hidden.py (konsensus v2 D38: mgła+bramka+migracja) 46.4s rc=0
✓ [CHAIN     ] contact_ref.py (P21/D45: nick+wallet-kontakt on-chain) 0.1s rc=0
✓ [CHAIN     ] ranks.py (D46: ranga z konsensusu, upgrade, 6 confs) 0.2s rc=0
✓ [CHAIN     ] ban_evt.py (D47: tombstone k-z-n 0x06, wykup D30) 0.2s rc=0
✓ [AI        ] ai_sentinel.py (M6: radar fail-open, wniosek+ban E2E) 0.9s rc=0
✓ [APP       ] exchange.py (giełda+PSC D32)            0.1s rc=0
✓ [APP       ] xmr_swap.py (D34)                       1.8s rc=0
✓ [TOOLS     ] self_attack.py (red team)                2.7s rc=0
```

## Kontrole statyczne: 21/21

```
✓ zero sekretów PEM w repo
✓ .gitignore: *.key
✓ .gitignore: secrets/
✓ .gitignore: *.iso
✓ .gitignore: __pycache__
✓ .gitignore: build/
✓ exec bit: os/spoof.sh
✓ exec bit: os/build_iso.sh
✓ exec bit: os/verify_iso.sh
✓ exec bit: os/includes.chroot/usr/local/bin/fenix-vpn
✓ exec bit: os/includes.chroot/usr/local/bin/fenix-netup
✓ docs: docs/decyzje_architektury.md
✓ docs: docs/threat_model.md
✓ docs: docs/ToS.md
✓ docs: docs/fnx_spec.md
✓ docs: docs/legal/FENIX_Dokument_PL.md
✓ docs: docs/legal/FENIX_Dokument_EN.md
✓ docs: docs/legal/FENIX_Dokument_RU.md
✓ GUI zaimplementowane (M7)
✓ AI-Sentry zaimplementowane (M6)
✓ git: drzewo czyste (poza regenerowanym raportem ataków)
```

# Błędy ZNALEZIONE w tej iteracji i NAPRAWIONE (każdy ma test-strażnik)

Zasada projektu: test najpierw łAPIE błąd, pokazujemy go wprost, naprawiamy, powtórzony przebieg. Poniżej pełna lista z tej rundy — z P0 na początku.

## [P0-KRYTYCZNY] net/fenix\_node.py

- Python -m net.fenix\_node ZAWSZE odpalał selftest — nawet gdy systemd wołało z flagami demona. Fenix-node.service na ISO nigdy nie wystartowałby prawdziwej sieci P2P.
- naprawa: poprawka kodu + strażnik w build\_iso selftest (iso już tego pilnuje)

## [WYSOKI] gui/pool\_panel.py (test)

- Selftest liczył nonce ręcznie i zderzył się z add\_tx (chain+mempool) — pokazało, jak ważny jest wzorzec nonce z ledgera, nie z pamięci.
- naprawa: poprawka w teście (produkcja była czysta)

## [WYSOKI] app/messenger.py

- E2E szyfrowało DH z kluczem NADAWCY zamiast ODBIORCY — nikt nie byłby w stanie przeczytać wiadomości (bezpiecznie FAIL, nie przeciek).
- naprawa: poprawka kodu; test E2E łapie

## [ŚREDNI] app/messenger.py

- LocalTransport.poll\_envs padał z KeyError, gdy odebrano przed pierwszą wiadomością (brak kolejki).
- naprawa: poprawka kodu (setdefault)

## [WYSOKI] chain/clsag.py

- MLSAG: sig.c0 = c[idx] podpisującego zamiast c[0] (konwencja z chain/ring\_sig) — weryfikator zaczynał od złego wyzwania: ŻADEN podpis by nie przeszedł.
- naprawa: poprawka kodu + wyzwanie z indeksem j (jedna szkoła)

## [WYSOKI] chain/clsag.py

- Podpis nad kanonem z placeholderów ≠ kanon liczony przez weryfikator (ki brał się do msg po podpisie).
- naprawa: poprawka kodu: ki PRZED policzeniem msg

## [NISKI] workspace

- Snapshot środowiska gubi exec-bity skryptów (5 plików) → kontrole exec-bit czerwone + build\_iso wymagające -x na fenix-gui padało.
- naprawa: rytuał chmod +x przed audytem/commitem (udokumentowane)

## [NISKI] gui/fenix\_gui.py (test)

- Test oczekiwał słowa 'IPC' w komunikacie o martwym VPN/miningu — kontrakt to (False, powód z demona).
- naprawa: poprawka w teście (kontrakt ujednolicony)

## [ŚREDNI (v002)] net/fenix\_node.py

- Mesh T\_MSG: dedup liczył blake2s(canon(env).encode()), a canon() zwraca BYTES — KAŻDA koperta wywracałaby relay (AttributeError).
- naprawa: poprawka kodu (bez .encode()); test 6 łapie E2E na 3 node'ach

## [P0-KRYTYCZNY (v003)] chain/ledger.py + tx\_ring.py (3a)

- SHIELD v1 z N wyjściami NIE miał limitu 1 — każdemu wyjściu kredytowano PEŁNE tx.amount: N× pieniądza z 1× spalania (cicha drukarnia FNX w dawnym poolu).
- naprawa: poprawka konsensusu: v1=DOKŁADNIE 1 wyjście (mempool+blok), suma==amount w v2; strażnik w chain/tx\_hidden.py (test 1)

## [ŚREDNI (v003)] chain/clsag.py (sanity)

- Build ringu v2 z ekonomicznie złym bilansem (1 FNX in vs 3 FNX out) wywracał sanity 'r\_in≠bilans' — udowodnił, że build sama pilnuje matematyki, zanim konsensus zobaczy podpis.
- naprawa: test 8 projektuje atak z POPRAWNĄ ekonomiką (złamanie tylko na regule v1-member)

## [P0-KRYTYCZNY (v004)] chain/clsag.py + tx\_hidden.py (3b)

- MLSAG zamykał bilans TYLKO dla 1 wejścia (z = r\_in - Σr\_out wymaga a\_i == Σout+fee PER WEJŚCIE). 3-wejściowy hidden-send z GUI (M7d) wywracał sanity 'bilans≠' — konsensus odrzucał legalne tx wielo-wejściowe (blokada funkcji, wykryta zanim trafiła na produkcję).
- naprawa: fix D41: pseudoOut per wejście jak w Monero (hC=commit(a\_i, r\_p\_i), Σr\_p == Σr\_out, D=C-pseudo\_i; konsensus Σpseudo == ΣC\_out + fee-H) + bilans JAWNIE liczbami w build (fail-fast). Test M7d 9/9 łapie regresję

## [ŚREDNI (v004)] chain/ledger.py (ekonomia D39)

- Przejście fee → KOPACZ bloku obnażyło 2 stare suity (tx\_ring, usernames) liczące salda bez zwrotu ownerskiej części fee kopaczowi (rozjezdne o 550 ppm/tx).
- naprawa: aserty D39-aware (+ fee\_split()[1] wraca do kopca); full\_audit 32/32 po zmianie

## [NISKI (v004)] gui/pool\_panel.py (test 9d)

- Assert formatu JSON oczekiwał spacji (separator bez niej) i salda bez coinbase kopacza (kop()) ładuje to samo konto).

→ naprawa: poprawka testu: format `""amt":1000000000"` + balans z `won9.txs[0].amount`

#### [ŚREDNI (v005)] app/messenger.py (ratchet kick)

- Próba kick przy nieznanym pub liczyła DH z DŁUGIEGO `x_priv` tożsamości zamiast z `ratchet-priv` tej rundy — rozmowa tam↔z powrotem nigdy by się nie zbiegła (pierwszy odbiór odpowiedzi umierał).

→ naprawa: poprawka na `DH(my_priv_rundy, ich_nowy_pub)`; test ping-pong+root-equality strzeże

#### [ŚREDNI (v005)] app/messenger.py (anty-replay v2)

- Surowy licznik `inner.seq` z v1 zabijał LEGALNE zamieszanie mesh (ramka 4 przed 3 odrzucana). Łańcuch ratchetu i tak deduplikuje zużyte mk.

→ naprawa: v2: dedup na łańcuchu, `inner.seq` = high-water UI; test skip-okna łapie

#### [NISKI (v005)] chain/ledger.py + net/fenix\_node.py (restart/pool)

- Edycja `chain.dat` skasowała metodę `snapshot()` (AttributeError w GUI, wykryte regresją `fenix_gui`); świeży proces messenger'a nie rozpoznawał aktualnej rundy pub (mapa `rk_pub`→`fp` jest RAM-only).

→ naprawa: `snapshot()` przywrócony + pełny `fenix_gui` w regresji; v2: próba kontynuacji łańcucha per kontakt zanim `kick-trial`; test 9d TF1-restart strzeże

#### [ŚREDNI (v006)] chain/ranks.py (6 confs)

- '6 potwierdzeń' policzone off-by-one między `active_rank` a `pending_rank` — aktywacja rozminęła się o blok (jedno liczyło z blokiem zakupu, drugie bez).

→ naprawa: ujednolicenie na standard N-confs (blok zakupu + N−1); test 2 pilnuje obu funkcji

#### [NISKI (v006)] chain/contact\_ref.py + chain/ranks.py (formaty/test)

- Sufit `ADDR_MAX=128` pękał przy prawdziwym `FNXS1` (~141 zn. — dopiero selftest E2E to zmierzył); test rang finansował 100 FNX przy dotacji 50 (brak środków — lekcja: testy też liczą saldo).

→ naprawa: `ADDR_MAX=160` + komentarz-dowy; funding testu koryguje przelew do 40 FNX

#### [ŚREDNI (v007)] chain/ban\_evt.py (kropka k-z-n)

- Selftest rejestrował attestorów w instancji `__main__` modułu, a ledger waliduje z instancji PAKIETU `chain.ban_evt` — kropka widniała jako pusta i legalny ban dostawał '0/3 ważnych podpisów' (bezpieczny fail-zamknięty, funkcja umarłaby na produkcji).

→ naprawa: rejestracja dev w instancji pakietu + komentarz w selfteście; ponadto kropka 3-różni+duplikat = LEGALNY kworum (poprawiono TEST, nie kod; prawdziwy atak-duplikat = 2+duplikat — odpada)

#### [ŚREDNI (v007)] ai/ai\_sentinel.py (bieg zalewu)

- Bieg flood'a startował od pierwszego przekroczenia progu — watch o ~60 s za późno; pakiet 2400 msg w 5 s udawał 'sustained flood' przez minutę okna.

→ naprawa: bieg od najstarszego wypełnionego wiaderka + strażnik anty-burst (ostatnie 10 s też gorące); testy 4/5 pilnują granic 60 s/5 min/10 min

#### [NISKI (v007)] ai/ai\_sentinel.py (test replay)

- Druga godzina replay-storm z odstępem 75 s to 48/h < 50: storm NIGDY by nie doszedł red (test bezsilnie zielony).

→ naprawa: odstęp 50 s = 72/h: druga odrębna godzina >50 odpala red 0x15

#### [NISKI (v007)] chain/ban\_evt.py (test wykupu)

- Funding testu (20 FNX) < wykup 100 FNX — 'brak środków' jak w D45 (lekcja powtórzona: wykupy też liczymy saldem).

→ naprawa: funding 24 bloki = 120 FNX > wykup; nonce wykupu = pierwszy tx konta (złapane tym samym błędem)

Wnioski z rundy: najgroźniejszy był P0 dispatch demona (sieć na ISO nigdy by nie wstała) — złapany przy okazji IPC, gdy CLI-demona sprawdzono „faktycznie” zamiast tylko selftestów. Lekcja: każdy plik-demon dostaje test uruchomienia z argv.

# Szyfrowanie — stan obrony (warstwa → algorytmy → wynik ataków)

## tożsamość+keystore

- Argon2id + KS1 AEAD; wabik panic
- ✓ self\_attack: brute/flip/spray — odparzone (27 odparzonych ogółem)

## ramki mesh

- X25519→HKDF→ChaCha20-Poly1305; nonce per SEQ; replay-window
- ✓ drut: 588 rekordów 4096B, ZERO markerów (frame/node selftest)

## camo/dnscamo

- rekordy stałe jak TLS; DNS base32 qname↔TXT
- ✓ loopback E2E; profile A/D przechodzą

## messenger E2E

- X25519 eph → HKDF('FNXM1') → ChaCha20-Poly1305 (AAD); podpis ed25519
- ✓ impersonacja: fp wykrywa; replay drop; flip bitu = bad\_aead

## pool/ring

- ed25519 LSAG + stealth ed25519 + Pedersen + Borromean 64-bit
- ✓ double-spend odrzut;  $\Sigma = \Sigma_{out} + fee$ ; druk FNX odrzut

## clsag 3b

- MLSAG 2-kolumnowy + blob AEAD (AAD=C)
- ✓ fee-klamstwo/ki-podmianka/rp-zepsuty → odrzuty; payload bez amt

## RNG platformy

- os.urandom
- ✓ test E: 512 KB — odparzone

## anti-forensic

- shred 2× nadpis (keystore/tofu/vpn.env), drop\_caches, sysrq
- ✓ panicd/dry+real shred — PASS

## Spoof (MAC / IP / inne) — analiza

os/spoof.sh: losowy MAC (02: lokalny+unicast, walidator odrzuca multicast/globalne), hostname fenix-<8hex>, machine-id 32hex — generowane PRZED siecią (network-pre.target), 200/200 unikalne w selftestach. Skutek: komputer z ISO nie da się skorelować po LAN (ta sama kawiarnia wczoraj/dziś to inna maszyna). Publiczny IP: tym nie zajmuje się spoof (to nie jego warstwa) — publiczny IP zasłania Mullvad WG (paranoia C z kill switchem) i/lub camo+mesh; bez VPN Fenix pokazuje IP hosta, co jest UDOKUMENTOWANĄ granicą profilu A/B, nie dziurą. DHCP/hostname UUID: losowane per boot; IPv6 EUI-64 dziedziczy z randomizowanego MAC.

# Problemy OTWARTE (uczciwie: co jeszcze nie działa / nie jest zweryfikowane)

## P2 [WYSOKA] sieć

- Seeds = placeholder DEV (203.0.113.x). Nowy node bez nich nie dołączy do realnej sieci.
- plan: właściciel:  $\geq 3$  VPS jako seed-nody; podmiana `/etc/fenix/seeds.list`

## P3 [WYSOKA] ISO/QA

- ISO nie bootowane w qemu/kvm (brak w sandboxie). `build_iso selftest` = 56/56 statycznie, ale to nie boot.
- plan: live-build + qemu na maszynie właściciela

## P5 [ŚREDNIA] ekonomia

- `BLOCK_REWARD/halving/ceny rang/UNBAN_FEE` = wartości robocze (`fnx_spec §12`); skarbiec = DEV wallet; wykup D30 bez escrow+oracle (backlog).
- plan: decyzja właściciela: emisja + zimny multisig D14 + escrow 2-z-3

## P7 [ŚREDNIA] giełda/swap

- Paysafe/monero-rpc gatewaye = kontrakty (NotImplemented); hosting deska poza kodem.
- plan: D32/D34: umowa + VPS

## P8 [ŚREDNIA] transport

- DNS-camo wymaga własnej domeny-mostu (zadanie właściciela). Loopback E2E działa.
- plan: rejestracja domeny + DnsBridgeServer

## P9 [ŚREDNIA] ISO

- dm-verity nie wdrożone.
- plan: backlog ISO: verity + podpis GPG offline

## P10 [ŚREDNIA] AI

- AI-Sentry MVP ISTNIEJE (v007/D48: radar fail-open 4 detektory, kapsuła RAM, wniosek→plomba E2E); brakuje: karmienie żywym ruchem z node'a, konsensus wielu sentineli, warstwa ML.
- plan: P26 (karmienie+enforce) → skala → warstwa 3 ML

## P11 [NISKA] krypto

- Rangeproof Borromean 6.3 KB/out (bloki cięższe).
- plan: bulletproofs backlog D31

## P12 [NISKA] transport

- `snowflake_fnx/onion-relay` (M2) nie istnieją (T\_MSG dla messengera już działa — D37, v002).
- plan: M2/M3 etapy

## P13 [NISKA] D28

- Avatar = tylko ref-hex; blob-store nie wdrożony.
- plan: M4+ blob-store

## P15 [INFO] Mullvad

- Prawdziwe up() (root+konto+sieć) nie testowane — offline 10/10.
- plan: QA na maszynie właściciela

## P17 [ŚREDNIA] GUI/QA

- WIDOK GUI (Tkinter) nie testowany pod X11/openbox — cała logika kontrolerów headless ✓ (`fenix_gui` 19/19), widok strukturalnie gotowy.
- plan: QA na ISO z openbox

## P24 [ŚREDNIA] messenger

- Ratchet FNX-R1 = MVP założony na FIFO per para (desync po utracie ramki = resync przy kolejnym kicku; ramki sprzed 2 rund = drop); concurrent-sessions jak w Signal = backlog. Grupy bez ratchetu.
- plan: sesje-wielo + grupy M4c

## P20 [NISKA] IPC/QA

- Żywy przebieg demon+GUI (2 procesy) nie testowany — IPC sprawdzony E2E z klasą Node i w testach CLI.
- plan: QA na ISO: `systemctl fenix-node + fenix-gui`

## P25 [WYSOKA] bezpieczeństwo

- Kropka attestorów `BAN_EVT` = pusta/dev-hook (żaden ban nie przejdzie bez wydania z zaszutą kropką — fail-zamknięty celowo); brak rotacji kropki i konsensusu wielu sentineli.
- plan: właściciel: zimne klucze attestorów (D14) + zaszucie wydaniem; potem `VOTE_EVT/VOTER` (PoU 30d)

## P26 [ŚREDNIA] AI/sieć

- Sentinel nie jest jeszcze karmiony żywym ruchem z `fenix_node` (`msg/bad_tag/replay/hello_fail`), throttle działa tylko rekomendacyjnie, mapowanie `fp→wallet` = operator.
- plan: wpięcie sentinel do `net/fenix_node` + enforce throttle per-peer

## Jak to uruchomić u siebie

```
selftest całości:      python3 tools/full_audit.py    (raport JSON → /tmp/fenix_audit.json)
red team:              python3 tools/self_attack.py
demon (2 procesy, test): python3 -m net.fenix_node --port 45992 --ipc /tmp/fenix.ipc
...a w drugim oknie GUI: python3 -m gui.fenix_gui --demo    (na maszynie z X11)
messenger demo:       python3 -m app.messenger --demo
klasa 3b demo:         python3 chain/clsag.py    (rdzeń – konsensus w follows)
budowa ISO (Debian):   bash os/build_iso.sh    → potem qemu/kvm smoke boot
regeneruj TEN raport:  python3 tools/make_rap001.py    (po full_audit)
```

### Czego NIE testowano (uczciwe granice tego raportu)

- widok GUI pod X11/openbox (logika kontrolerów 15/15 headless PASS; widok = P17)
- boot ISO w qemu/kvm + live-QA Mullvad z kontem (P3/P15)
- pełny mesh demon↔demon z GUI dwoma procesami (P20) — komponenty E2E tak
- obciążenia >sandbox: kopie 1000 node'ów, fragmentacja sieci, czas godzin (M9 QA)

Hasło raportu: projekt NIE deklaruje 'brak dziur 0day' — deklaruje i udowadnia, że (a) każdy obronny mechanizm jest testowany SAM przeciw sobie, (b) znalezione błędy są naprawiane z testem-strażnikiem, (c) lista problemów otwartych jest jawna, bez makijażu. To jest sposób, w jakim projekt zasługuje na zaufanie.